Campus IT Service Desk Handover Lab

Lab 2 — Terminal Tools · AlgoUniversity IT Support Centre

Scenario and Setup

Your role

You are the incoming technician at AlgoUniversity's IT Support Centre.

The outgoing shift left three problems in the records:

- The handover note has the **wrong room and start time** (copied from an old draft).
- The ticket ownership log has **duplicate entries** from a merged export.
- The working task list is the **old version**; the approved replacement was never applied.

Your job is to inspect, repair and validate all three problems using seven terminal tools.

Tools covered

Exp	Tool	Skill
1	`history`	Recall, search and reuse commands
2	`nano`	Edit and create plain-text files
3	`less`	Navigate long files; search by keyword
4	`sort`	Sort lines alphabetically or numerically
5	`uniq`	Remove duplicates; count groups
6	`cut`	Extract fields from delimited records
7	`diff`	Compare versions; guide repairs
—	Mission	Integrate all seven tools on fresh evidence

Starting up

```
bash start.sh
```

When prompted, **enter your own name or student number** as your ID.

Use only letters, digits and hyphens — no spaces. Examples:

```
your-name        (e.g.  alex-smith)
your-student-number   (e.g.  s1234567)
student-demo     (if your instructor told you to use this)
```

Run the `cd` command that setup prints. **Paste it exactly** — do not type it by hand.

Then confirm your location and read your identity file:

```
pwd
cat identity.txt
```

`identity.txt` contains four values. **Write down ROOM and START** — you will use them in Experiment 2.

> **Rule:** Begin every experiment from `lab-work/lab2-/` (e.g. `lab-work/lab2-student-demo/`).

> Never edit files in `evidence/` or `mission/` — they are read-only originals.

> Redirect with `>` always to a *different* file from the one you are reading.

> **Paste commands one at a time** — do not paste multiple lines at once.

Experiment 1 — history

`history` prints a numbered list of every command you have run. You can filter it, jump back to an earlier command interactively, accept it for editing, or cancel without running it. This avoids retyping long commands and catches mistakes early.

Command / key	Action	
`history 10`	Show the last 10 entries	
`history | grep word`		Show only entries containing *word*
`Ctrl+R`	Open reverse search; type to find a match	
`Ctrl+R` again	Jump to the next (older) match	
`Right Arrow`	Accept the found command for editing	
`Enter`	Accept and run immediately	
`Ctrl+G`	Cancel the search, return to empty prompt	

1A — Build a history trail.

Run all three commands below. They create the trail you will search in 1B–1D.

```
grep SHIFT_ID identity.txt
grep ROOM identity.txt
grep START identity.txt
```

Note the values printed for ROOM and START.

1B — Print and filter.

```
history 10
history | grep grep
```

How many lines contain the word `grep`? What line number is the earliest?

1C — Ctrl+R search, accept and edit.

Press `Ctrl+R` and type `ROOM`. The shell shows `grep ROOM identity.txt`.

Press `Ctrl+R` again to move to an older match.

Press `Right Arrow` to bring the command to the prompt.

Edit it to read `grep START identity.txt` and press `Enter`.

1D — Cancel a search.

Press `Ctrl+R`, start typing `sev`, then press `Ctrl+G`.

Nothing runs. The prompt returns blank.

> **Connection:** The ROOM and START values from 1A are the corrections needed

> in Experiment 2.

Experiment 2 — nano

`nano` is a terminal text editor. Key shortcuts appear at the bottom of the screen.

`^` always means `Ctrl`.

Key	Action
`Ctrl+O`	Save (write out) — press `Enter` to confirm filename
`Ctrl+X`	Exit
Arrow keys	Move cursor

2A — Edit an existing file.

```
nano work/handover-note.txt
```

The file has four lines. Two contain placeholder values:

```
Room: ITSD-00      ← replace with your ROOM value
Start: 00:00       ← replace with your START value
```

Navigate with arrow keys. Replace both wrong values. Save with `Ctrl+O`, `Enter`.

Exit with `Ctrl+X`.

2B — Reopen and verify.

```
cat work/handover-note.txt
nano work/handover-note.txt
```

Confirm both lines are correct, then exit with `Ctrl+X`.

2C — Create a new file.

```
nano work/summary.txt
```

The file does not exist. nano opens an empty editor. Type one sentence describing the shift handover purpose. Save with `Ctrl+O`, `Enter`, `Ctrl+X`.

```
cat work/summary.txt
```

> **Connection:** The shift instruction in Experiment 3 confirms the room and
>
> time you just saved.

Experiment 3 — less

`less` displays a long file one screen at a time without modifying it.

Key	Action
`Space`	Forward one screen
`b`	Backward one screen
`g` / `G`	Jump to start / end
`/text` + `Enter`	Search forward
`n` / `N`	Next / previous match
`q`	Quit

3A — Navigate.

```
less evidence/incident-log.txt
```

Press `Space` several times, then `b`, then `G`, then `g`. Press `q` to quit.

3B — Search for your shift section.

Reopen the file. Type `/SHIFT LOG` and press `Enter`.

- First match: **SHIFT LOG - ARCHIVED** — outdated. Do not use it.
- Press `n` → **SHIFT LOG - ACTIVE SHF-xxxxxx** — verify the code matches

your SHIFT_ID from `identity.txt`. The next line is your shift instruction.

- Press `n` → **SHIFT LOG - OTHER SHIFT** — wrong shift. Press `N` to return to ACTIVE.

Read the instruction line. Press `q`.

3C — Save the shift instruction.

```
nano work/shift-instruction.txt
```

Type the exact sentence from the ACTIVE section. Save and exit.

Verify the room and time match `work/handover-note.txt`.

> **Connection:** You now have the correct names and times confirmed from two
>
> independent sources. Experiments 4–5 work on the ticket ownership log.

Experiment 4 — sort

`sort` reads lines and writes them in order. By default the order is alphabetical.

Flag	Meaning
(none)	Alphabetical (text) sort, ascending
`-n`	Numeric sort
`-nr`	Numeric, descending (largest first)

4A — Alphabetical sort and save.

```
sort evidence/ticket-owners.txt
sort evidence/ticket-owners.txt > work/owners-sorted.txt
cat work/owners-sorted.txt
```

How many lines? (Answer: 8 — duplicates are preserved; sort does not remove them.)

4B — Text sort vs numeric sort.

```
cat evidence/severity-scores.txt
sort evidence/severity-scores.txt
sort -n evidence/severity-scores.txt
```

Observation: Why does text sort put `100` before `2`?

Hint: text sort compares character by character. `1` comes before `2` alphabetically, so `100` sorts before `2` regardless of its numeric value.

4C — Numeric descending and save.

```
sort -nr evidence/severity-scores.txt > work/scores-desc.txt
cat work/scores-desc.txt
```

First line must be `100`; last line must be `2`.

> **Connection:** Experiment 5 uses `evidence/ticket-owners.txt` —
>
> the same file sorted here.

Experiment 5 — uniq

`uniq` removes **adjacent** duplicate lines. Lines that repeat but are not next to each other are left untouched. This means you must sort first.

Flag	Meaning
(none)	Remove adjacent duplicates
`-c`	Prefix each unique line with its occurrence count

5A — Adjacent duplicates only.

```
cat evidence/ticket-owners.txt
uniq evidence/ticket-owners.txt
```

Count the output lines. Are all duplicates gone, or only some?

Find two identical names on consecutive lines (adjacent) and two with other names between them (separated). `uniq` removes only the adjacent pair.

5B — Correct order: sort then uniq.

```
sort evidence/ticket-owners.txt | uniq > work/unique-owners.txt
cat work/unique-owners.txt
```

How many lines? (Answer: 4 — all duplicates removed.)

5C — Wrong order comparison.

Predict what `uniq | sort` will produce. Then try it:

```
uniq evidence/ticket-owners.txt | sort
```

Command	Output lines	Duplicates?	
`uniq |	sort`	7	Yes — three names still repeat
`sort |	uniq`	4	No

`uniq | sort` removes only the one adjacent pair first, then sorts — but the separated duplicates now sit next to each other **after** `uniq` has already finished. Always use `sort | uniq`.

5D — Count occurrences.

```
sort evidence/ticket-owners.txt | uniq -c > work/owner-counts.txt
cat work/owner-counts.txt
```

Do the counts sum to 8? (They must — that is the total number of lines in the source file.)

> **Connection:** Experiment 6 extracts the same names from `evidence/tickets.txt`.

Experiment 6 — cut

`cut` extracts specific fields from each line of a delimited file.

Field numbering starts at 1. `cut` preserves the original row order.

Flag	Meaning
`-d "char"`	Set the field delimiter
`-f N`	Extract field N
`-f M,N`	Extract fields M and N

`evidence/tickets.txt` uses `|` as a delimiter. Because `|` is a shell pipe character, it must be quoted. Each record has three fields:

```
T01|Name|Category
```

6A — Extract owner names (field 2).

```
cat evidence/tickets.txt
cut -d "|" -f 2 evidence/tickets.txt > work/ticket-owners-list.txt
cat work/ticket-owners-list.txt
```

Compare the output to `evidence/ticket-owners.txt`.

They should be identical — both files come from the same source export.

6B — Extract ID and category (fields 1 and 3).

```
cut -d "|" -f 1,3 evidence/tickets.txt > work/id-category.txt
cat work/id-category.txt
```

The output delimiter is still `|` — `cut` cannot change it.

6C — Count tickets by category (full pipeline).

```
cut -d "|" -f 3 evidence/tickets.txt | sort | uniq -c > work/category-counts
cat work/category-counts.txt
```

- `cut` extracts the category field from each row.
- `sort` groups identical categories together.
- `uniq -c` counts each group.

Do the counts sum to 8?

> **Connection:** Experiment 7 compares and repairs the task list —

> the final piece of evidence needed for the handover.

Experiment 7 — diff

`diff` compares two files line by line. Identical files produce no output.

The first argument is always the **older** file; the second is the **newer** file.

Marker	Meaning
`Nc,Nd`	Lines N–N in file 1 **c**hanged or were **d**eleted
`<`	Line from file 1 (older)
`---`	Separator between old and new
`>`	Line from file 2 (newer)
`c`	Both sides changed (replaced)
`d`	Line deleted from file 1
`a`	Line added in file 2

7A — Identical files produce no output.

Predict: what does `diff` print when both arguments are the same file?

```
diff evidence/tasklist-approved.txt evidence/tasklist-approved.txt
```

No output confirms the files are identical. This is how you verify a repair.

7B — Compare before and approved versions.

```
cat evidence/tasklist-before.txt
cat evidence/tasklist-approved.txt
diff evidence/tasklist-before.txt evidence/tasklist-approved.txt
```

You will see:

```
3,4c3,4
< 09:30 Ticket triage
< 10:00 Lunch cover
---
> 10:00 Ticket triage
> 11:00 Shift handover
```

`3,4c3,4` means lines 3–4 in file 1 were replaced by lines 3–4 in file 2.

`<` shows the old lines; `>` shows the approved replacements.

Questions:

- How many lines changed?
- Which task had its time shifted?
- Which task was replaced entirely?

7C — Repair the working copy.

```
nano work/tasklist.txt
```

Use the `>` lines from the diff as your guide. Apply both changes.

Save with `Ctrl+O`, `Enter`, `Ctrl+X`. Verify:

```
diff work/tasklist.txt evidence/tasklist-approved.txt
```

No output means the repair is complete. If diff shows output, reopen nano,

fix the remaining difference, and re-run.

Independent Mission

You have practised all seven tools. The mission uses a **fresh set of evidence**

in `mission/`. No step-by-step commands are given — apply what you have learned.

Deliverables

Produce these five files in `work/mission/`:

File	Contents	
`handover-note.txt`	The corrected four-line handover note	
`open-tickets.txt`	Alphabetical list of distinct ticket owners	
`category-counts.txt`	Ticket count per category (`sort \	uniq -c` format)
`tasklist-diff.txt`	Saved diff of old vs approved schedule	
`tasklist.txt`	Working schedule updated to match the approved version	

Mission evidence (read-only)

File	Description		
`mission/brief.txt`	Long briefing document — contains the approved handover note		
`mission/tickets.txt`	Pipe-delimited: `ID\	owner\	category`
`mission/schedule-before.txt`	Old schedule		
`mission/schedule-approved.txt`	Approved schedule		

Working copies already exist in `work/mission/` (start from these).

Steps

Step 0 — Survey your history.

Run `history 10`. You will see `diff`, `cut`, `sort`, `uniq` and `nano`

from Experiments 1–7. You will use `Ctrl+R` in Step 7 to recall one of them.

Step 1 — Find the approved handover note (`less`)

Open `mission/brief.txt`. Search for `HANDOVER - APPROVED`.

The brief contains a `HANDOVER - OLD` section — skip it.

Find the section whose shift code matches your `identity.txt`. Below its

instruction line are the four lines you need. Note them. Quit.

Step 2 — Repair the handover note (`nano`)

Edit `work/mission/handover-note.txt`. Fix the Room and Start values.

Save, exit, verify with `cat`.

Step 3 — List open ticket owners (`cut`, `sort`, `uniq`)

From `mission/tickets.txt`, extract field 2, deduplicate and sort.

Save to `work/mission/open-tickets.txt`.

Step 4 — Count tickets by category (`cut`, `sort`, `uniq -c`)

From `mission/tickets.txt`, extract field 3, count each category.

Save to `work/mission/category-counts.txt`.

Step 5 — Save the schedule comparison (`diff`)

Compare `mission/schedule-before.txt` (first) and `mission/schedule-approved.txt`.

Save to `work/mission/tasklist-diff.txt`.

Read the saved file. What marker letters appear — `c`, `d` or `a`?

Compare them to the markers in Experiment 7B. The mission schedule changed

differently: one line was deleted (`d`) and one was added (`a`).

Step 6 — Repair the working schedule (`nano`, `diff`)

Edit `work/mission/tasklist.txt` using the saved diff as your guide.

Verify:

```
diff work/mission/tasklist.txt mission/schedule-approved.txt
```

No output means success.

Step 7 — Recall from history (`Ctrl+R`, `Right Arrow`)

Press `Ctrl+R`, type `diff`. Step through matches until you find the verification diff from Step 6. Press `Right Arrow`. Edit the filenames to compare `mission/schedule-before.txt` against `mission/schedule-approved.txt` again. Run it. The output should match `work/mission/tasklist-diff.txt`.

Validation

Run from your lab folder:

```
python3 ../../validation/check.py
```

Result	Meaning
`OK`	File matches expected output
`FAIL`	File is missing, wrong or malformed — a corrective command is shown

Goal: all checks pass. Re-run after each fix.

If `Original evidence files preserved` shows FAIL, you edited a file in `evidence/` or `mission/`. Ask your instructor for a fresh attempt with a new ID.

> `history`, `nano` and `less` require a live demonstration.

> A full checker pass does not replace the instructor walkthrough.

Command Quick Reference

```
# Setup
bash start.sh                              # create your attempt
cat identity.txt                           # view your shift details

# history
history 10                                 # last 10 commands
history | grep word                        # filter by keyword
Ctrl+R  →  type  →  Ctrl+R                # reverse search
Right Arrow                                # accept match for editing
Ctrl+G                                     # cancel search

# nano
nano filename                              # open or create
Ctrl+O  Enter                              # save
Ctrl+X                                     # exit

# less
less filename                              # open for reading
Space / b                                  # forward / backward
g / G                                      # start / end
/text  Enter  →  n / N                     # search, next, previous
q                                          # quit

# sort
sort file > output                         # alphabetical, save
sort -n file                               # numeric
sort -nr file > output                     # numeric descending, save

# uniq
sort file | uniq > output                  # distinct lines
sort file | uniq -c > output               # count groups

# cut
cut -d "|" -f 2 file > output              # extract field 2
cut -d "|" -f 1,3 file > output            # extract fields 1 and 3
cut -d "|" -f 3 file | sort | uniq -c > output

# diff
diff before.txt after.txt                  # compare (older file first)
diff before.txt after.txt > saved.txt      # save comparison

# checker  (run from inside your lab folder)
```

```
python3 ../../validation/check.py        # e.g. from lab-work/lab2-student-d

# reset (if needed — replace student-demo with your own ID)
bash ../../scripts/reset_work.sh student-demo   # prompts for yes; never tou
```